

3. Decision Tree Algorithm

```
import pandas as pd
import math
```

```
data = pd.read_csv(r"D:\4 SLOT\ITA0606 ML\lab\decisiontreeexample.csv")
```

```
def entropy(target):
    values = target.value_counts()
    total = len(target)
    e = 0
    for v in values:
        p = v / total
        e -= p * math.log2(p)
    return e
```

```
def information_gain(data, attribute, target):
    total_entropy = entropy(data[target])
    values = data[attribute].unique()
    weighted_entropy = 0

    for value in values:
        subset = data[data[attribute] == value]
        weighted_entropy += (len(subset) / len(data)) * entropy(subset[target])

    return total_entropy - weighted_entropy
```

```
def id3(data, attributes, target):
    if len(data[target].unique()) == 1:
        return data[target].iloc[0]

    if len(attributes) == 0:
        return data[target].mode()[0]

    gains = [information_gain(data, attr, target) for attr in attributes]
    best = attributes[gains.index(max(gains))]
```

```
tree = {best: {}}
```

```
for value in data[best].unique():
    subset = data[data[best] == value]

    if subset.empty:
        tree[best][value] = data[target].mode()[0]
    else:
```

```

        remaining = [a for a in attributes if a != best]
        tree[best][value] = id3(subset, remaining, target)

    return tree

attributes = list(data.columns[:-1])
target = "Result"

tree = id3(data, attributes, target)

def display(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + str(tree))
        return

    for root in tree:
        print(indent + root)
        for value, child in tree[root].items():
            print(indent + " |—" + str(value))
            display(child, indent + " | ")

display(tree)

sample = {
    "Attendance": "High",
    "StudyHours": "Medium",
    "Assignment": "Good",
    "InternalMarks": "High",
    "Project": "Complete"
}

def predict(tree, sample):
    while isinstance(tree, dict):
        root = next(iter(tree))
        value = sample[root]
        tree = tree[root][value]
    return tree

print("\nNew Sample Classification:", predict(tree, sample))

```

OUTPUT:

Project

|— Complete
| Pass
|— Incomplete
| Fail

New Sample Classification: Pass